Software
Requirments
Specification

GesturaAI

Sign Language Translation & Learning System

Rawaha Zubair Khan
Umama Saif

1. Introduction

GesturaAI: Sign Language Translation & Learning System is designed to bridge communication gaps between the deaf or hard-of-hearing community and individuals who rely on spoken language. The system aims to provide a smooth accessible method for translating hand gestures into meaningful text and speech while also supporting learning for users who want to understand or practice sign language. By combining recognition, translation, and educational components in a single environment, the project focuses on making everyday communication easier, faster, and more inclusive.

The system captures live hand gestures, processes them through a structured recognition pipeline, and converts them into understandable language output. This includes identifying hand shapes, movement patterns, and gesture sequences, ensuring that translation is accurate and contextually relevant. Along with real-time translation, GesturaAI also provides a learning module that helps users practice signs, receive feedback, and gradually improve their understanding of sign language. This dual purpose strengthens the system's value by supporting both communication and education.

Overall**,** GesturaAI delivers a user-friendly solution that aligns with modern accessibility goals. The project emphasizes reliability, ease of use, and practical learning support ensuring the system can assist everyday users, learners, and individuals who interact with sign language regularly. Through its combined translation and learning capabilities, GesturaAI aims to make sign language more approachable for everyone.

2. Project Scope

The scope of *GesturaAI: Sign Language Translation & Learning System* covers the complete process of recognizing hand gestures, translating them into meaningful text or speech, and supporting users in learning and practicing sign language. The project includes all essential components required to capture gestures, classify them accurately, and provide clear outputs that reflect the intended meaning of the signs. It also extends to offering an interactive learning environment where users can explore sign vocabulary, practice gestures, and receive guided feedback to improve their skills.

The system focuses on a structured workflow that begins with gesture capture through a camera interface followed by preprocessing to enhance image quality and isolate hand regions. It further incorporates feature extraction, classification, and translation mechanisms to ensure gestures are understood correctly. In addition to translating the learning module forms an important part of the scope, enabling users to view gesture demonstrations, attempt signs, and track their progress over time.

This project does not limit itself to basic gesture detection. Instead, it includes all functional and technical aspects necessary for gesture recognition, multilingual translation output, performance evaluation, user interaction, and learning support. The scope also involves system integration, dataset handling, and interface development to provide a smooth and complete user experience. By covering both translation and education within a unified system, GesturaAI ensures that every feature contributes toward making sign language communication and learning accessible, reliable and practical for everyday use.

3. Feasibility Analysis

The feasibility analysis evaluates whether *GesturaAI: Sign Language Translation & Learning System* can be developed, implemented, and practically used within realistic technical, operational, financial and time-related limits. The project includes complete gesture recognition, translation, and learning features; therefore, each feasibility dimension focuses on determining if these components can function reliably and fit the intended environment.

3.1 Technical Feasibility

This section evaluates whether the required technologies, tools, and development environment support the full implementation of GesturaAI.

a. System Components

- Gesture Capture
- Preprocessing
- Feature Extraction
- Classification
- Translation Module
- Learning Module

All components operate using standard and accessible software tools without requiring specialized equipment.

b. Hardware Requirements

- Standard desktop or laptop
- Built-in or external webcam

No additional sensors or high-end devices are needed.

c. Software Requirements

- Common programming languages and libraries for:
    - Machine learning
    - Computer vision
    - Model training

    - User interface development

- **MERN Stack (MongoDB, Express.js, React.js, Node.js)** for building the full application interface, managing user interactions, storing learning progress and handling translation outputs.

d. Machine Learning Support

- Models for recognizing hand shapes, motion patterns and gesture sequences
- Training possible with available gesture datasets
- Optimization ensures accurate and reliable results

e. Development Environment

- Works with widely accessible tools
- Does not require high-performance computing
- Suitable for academic-level or prototype-level development

Overall, *GesturaAI* is technically feasible because every component from gesture capture to translation can be implemented using standard hardware, open-source frameworks, and development tools. Integrating the MERN stack further strengthens interface development and system management without increasing complexity.

3.2 Operational Feasibility

This section evaluates whether users can operate the system smoothly in real-life usage.

a. Ease of Use

- User simply performs gestures in front of the webcam
- System automatically captures, processes and translates input
- No complex setup required

b. Practical User Workflow

- Clear instructions
- Visual demonstrations in the learning module
- Real-time feedback
- Smooth translation response time

c. User Groups Supported

- Individuals learning sign language
- People interacting with the deaf or hard-of-hearing community
- General users interested in gesture-to-text interaction

d. Adoption

- Intuitive workflow
- No additional training required
- Suitable for daily use or educational purposes

Operational requirements are simple and realistic, making adoption smooth for all intended user groups.

3.3 Economic Feasibility

This section evaluates whether the system can be built and maintained affordably.

a. Cost Assumptions

Component	Assumed Cost
Development machine (existing laptop)	Rs. 0 (already available)
Webcam (if needed)	Rs. 2,500
Software tools / libraries	Rs. 0 (open source)
Model training (electricity + compute hours)	Rs. 1,500
Internet + utilities	Rs. 1,000
Miscellaneous development expenses	Rs. 1,000

b. Estimated Total Development Cost

Total = Rs. 2,500 + 1,500 + 1,000 + 1,000 = Rs. 6,000

c. Maintenance Costs

- Occasional updates to improve recognition accuracy

- Adding new gestures to the learning library
- No recurring licensing fees
- Minimal long-term operating expenses

The project is economically feasible because it relies entirely on existing hardware and free software tools. The total estimated cost remains low and manageable for an academic-level system.

3.4 Schedule Feasibility

This section examines whether the system can be completed within a reasonable timeframe. A structured schedule provides clarity, including a timeline table.

a. Development Phases

- Dataset Handling
- Preprocessing
- Model Training
- Interface Development
- Translation Module
- Learning Module
- Testing & Integration

b. Timeline Table

Phase	Duration	Description
Dataset Handling	Week 1	Collect and organize gesture datasets

Preprocessing	Week 2	Implement image cleaning, background removal and isolation
Feature Extraction	Week 3	Extract meaningful gesture features
Model Training	Week 4-5	Train recognition models, tune accuracy
Interface Development (MERN)	Week 6-7	Build user interface + learning module
Translation Module	Week 8	Map recognized gestures to text/speech
Learning Module Completion	Week 9	Add demonstrations, progress tracking
Testing & Integration	Week 10	Combine all modules into a full system
Final Refinements	Week 11	Polish performance and user flow

3.5 Resource Feasibility

This section evaluates the hardware, software, human, and data resources required.

a. Hardware Resources

- Standard computer or laptop
- Webcam for gesture capture
- Basic storage for datasets
- Moderate processing power for model training

b. Software Resources

- Machine learning libraries
- Computer vision tools

- MERN Stack environment
- Open-source utilities for dataset handling and interface development

c. Human Resources

- Small development team
- Basic machine learning and software development skills
- Ability to work in parallel on modules (gesture capture, preprocessing, recognition, translation, learning UI)

d. Data & Storage

- Gesture datasets requiring moderate memory
- Trained models with small to medium storage footprint

All required resources for hardware, software, personnel, and datasets are easily available and manageable. The project is strongly feasible in terms of resource availability.

4. Conclusion

GesturaAI: Sign Language Translation & Learning System is a technically sound, operationally practical, economically reasonable and resource efficient project. All features outlined in the project of gesture recognition, translation and learning modules are fully in scope and achievable within the proposed timeline and resource availability. The system is designed to be user friendly, reliable, and adaptable to multiple user groups from individuals learning sign language to people communicating with the deaf or hard-of-hearing community.

Overall, the feasibility analysis confirms that *GesturaAI* can be successfully developed, implemented, and adopted without major obstacles, making it a viable and impactful solution for sign language translation and learning.

Requirements Specification

Functional & Non - Functional Requirements

5. Requirements Specification

This section defines the functional and non-functional requirements for *GesturaAI: Sign Language Translation & Learning System.* All features listed in the project scope are included in the scope.

5.1 Functional Requirements

5.1.1 Module 1 - Sign Language Recognition (Sign to Text/Speech)

1. **Real Time Sign Detection**
   Access: All users
   Description: System captures live hand and body movements through webcam and identifies signs instantly.

2. **Gesture Recognition**
   Access: All users
   Description: Detects structured sign gestures, hand shapes and movements to translate them into text/speech.

3. **Fingerspelling Detection**
   Access: All users
   Description: Identifies alphabet-based hand gestures used for names, proper nouns and spelling words.

4. **Facial Expression / NMF Processing**
   Access: All users
   Description: Recognizes facial cues that modify meaning in sign language for more accurate translation.

5. **Multimodal Input (Hand + Face + Body)**

   Access: All users

   Description: Processes combined visual inputs to improve translation accuracy.

6. **Output in Text & Speech**

   Access: All users

   Description: Converts detected signs into readable text and clear voice output.

5.1.2 Module 2 - Text/Voice to Sign Translation

1. **3D Animated Avatar Signing**

   Access: All users

   Description: Displays animated avatar performing the translated signs.

2. **Multi-Language Sign Support (ASL/BSL/PSL etc.)**

   Access: All users

   Description: Allows users to select their preferred sign language for translation output.

3. **Enhanced Avatar Facial Expressions**

   Access: All users

   Description: Avatar uses accurate facial expressions to reflect correct sign meaning.

4. **Emotional Tone Responsiveness (Optional)**

   Access: All users

   Description: Avatar adjusts expressiveness based on detected emotional tone in text or voice.

5. **Text/Voice Input Support**

   Access: All users

   Description: Accepts user input via typing or microphone for translation to sign

language.

5.1.3 Module 3 - Accessibility Tools

1. **Instant Text Reader**

   Access: All users

   Description: Converts any displayed text into sign language via avatar.

2. **Translation Mode Selection (Sign ↔ Text ↔ Speech)**

   Access: All users

   Description: Users can freely switch between translation directions based on need.

3. **Offline Mode**

   Access: All users

   Description: Allows translation and learning features to work without internet.

4. **Customizable Avatars**

   Access: All users

   Description: Users may personalize avatar appearance for comfort and accessibility.

5. **Sign Dictionary**

   Access: All users

   Description: Provides a searchable list of signs with animations and examples.

6. **Notation Support (HamNoSys / SiGML)**

   Access: Advanced users / learners

   Description: Displays standardized sign notation for learning and technical reference.

5.1.4 Module 4 - Learning & Educational Mode

1. **Step-by-Step Avatar Tutorials**

   Access: Learners

   Description: Guides users through gestures with slow, beginner friendly animations.

2. **Fingerspelling Lessons**

   Access: Learners

   Description: Teaches alphabet-based signing with practice examples.

3. **Practice with Feedback**

   Access: Learners

   Description: System evaluates user performed signs and provides accurate feedback.

4. **AI Tutor Mode**

   Access: Learners

   Description: Offers suggestions, hints and corrections based on user performance.

5. **Difficulty Levels**

   Access: Learners

   Description: Allows users to advance from beginner to intermediate to advanced training.

5.1.5 Module 5 - Responsible AI & Ethical Features

1. **Cultural Sign Variation Support**

   Access: All users

   Description: Allows selection/customization of regional sign variations.

2. **Bias Detection & Correction**

   Access: System level

Description: Identifies potential inconsistencies and ensures fair, accurate translation.

3. **Community Feedback Loop**

   Access: All users

   Description: Users can report errors, suggest improvements and refine system output.

4. **Translation Confidence Indicators**

   Access: All users

   Description: Displays the confidence level of each translation for transparency.

5. **Data Privacy Protections**

   Access: All users

   Description: Ensures secure handling of visual, audio and text data during translation.

5.2 Non-Functional Requirements

5.2.1 Product Requirements

1. **Performance**

    - Real-time translation with minimal delay
    - Avatar animations must run smoothly
    - Efficient processing of multimodal inputs

2. **Reliability**
    - System must provide consistent translation outputs
    - Fail-safe fallback when gestures are unclear
    - Stable performance in varying lighting conditions

3. **Usability**
    - Simple and clean interface
    - Easy navigation for Deaf and hearing users
    - Tutorials and instructions must be accessible

4. **Portability**
    - Works on standard laptops or mobile devices with a camera
    - Compatible with common operating systems

5. **Efficiency**
    - Optimized ML models for fast inference
    - Lightweight UI built on MERN stack
    - Low memory overhead during recognition

6. **Accuracy**

- High gesture recognition accuracy
- Precise fingerspelling detection
- Correct mapping of avatar signs to input meaning

5.2.2 Organizational Requirements

1. **Development Standards**

    - MERN stack to be used for all web-based components
    - Modular development for easy maintenance
    - Code documentation required for each module

2. **Security Policies**
    - User data must be anonymized when stored
    - Access control for sensitive features
    - Standard coding practices for secure development

3. **Quality Assurance Requirements**
    - Unit testing for ML components
    - UI/UX testing for accessibility
    - Regular model calibration to maintain accuracy

5.2.3 External Requirements

1. **Accessibility Standards**

    - Interface must support accessibility guidelines (contrast, subtitles, readable text)
    - Avatar animations must follow sign clarity conventions

2. **Privacy & Data Protection**
    - User visual/audio data must not be misused
    - Clear consent for camera/microphone usage
    - No unauthorized storage of biometric data

3. **Ethical Requirements**

    - Transparent translation confidence indicators
    - Respect cultural variations and community standards
    - No biased or discriminatory model outputs

List of Abbreviations

AI: Artificial Intelligence

ASL: American Sign Language

BSL: British Sign Language

PSL: Pakistan Sign Language

MERN: MongoDB, Express.js, React.js, and Node.js

ML: Machine Learning

UI/UX: User Interface and User Experience

NMF: Non-Manual Features (facial expressions or body movements used in signing)

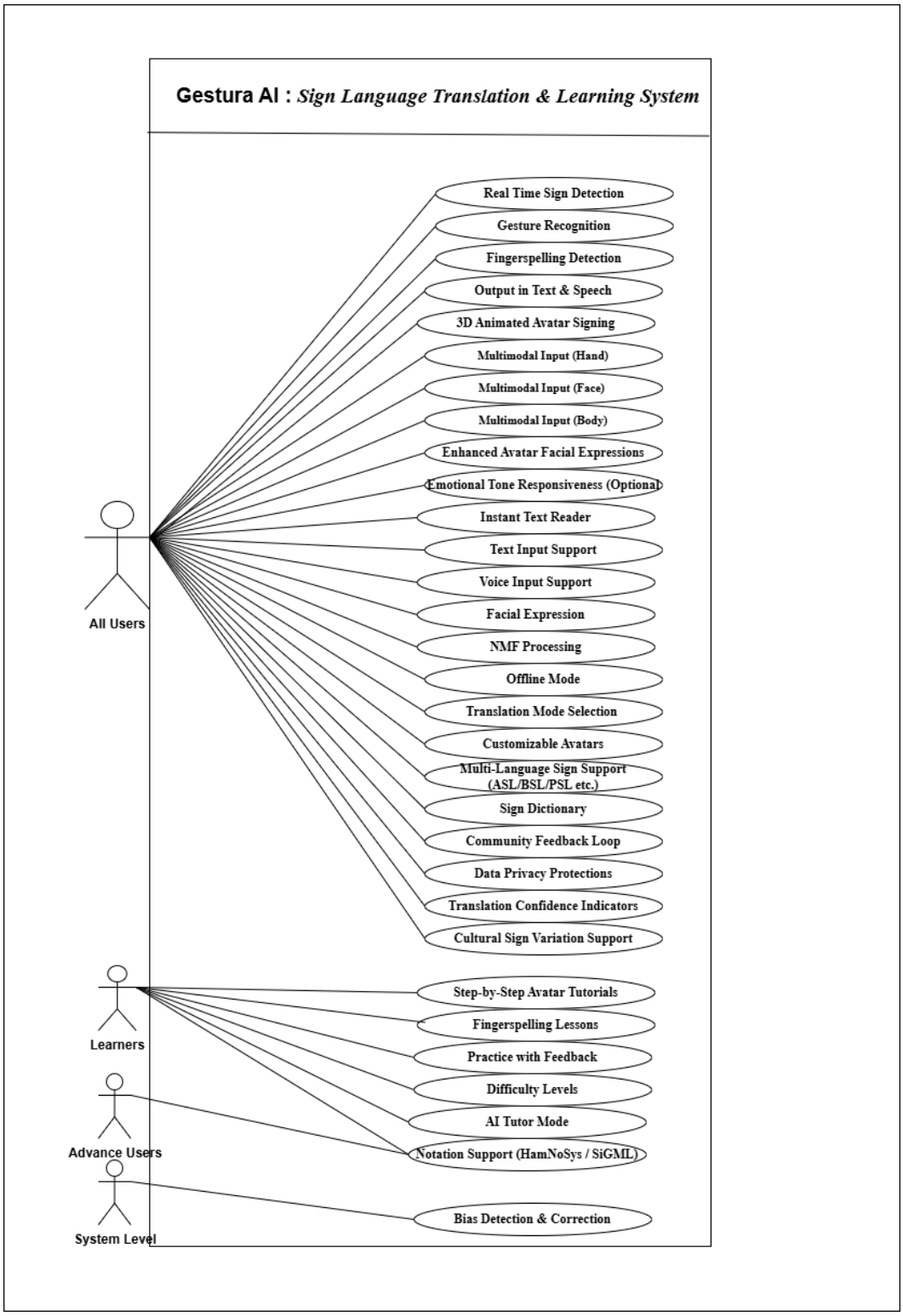
HamNoSys: Hamburg Notation System (a way to write down signs using symbols)

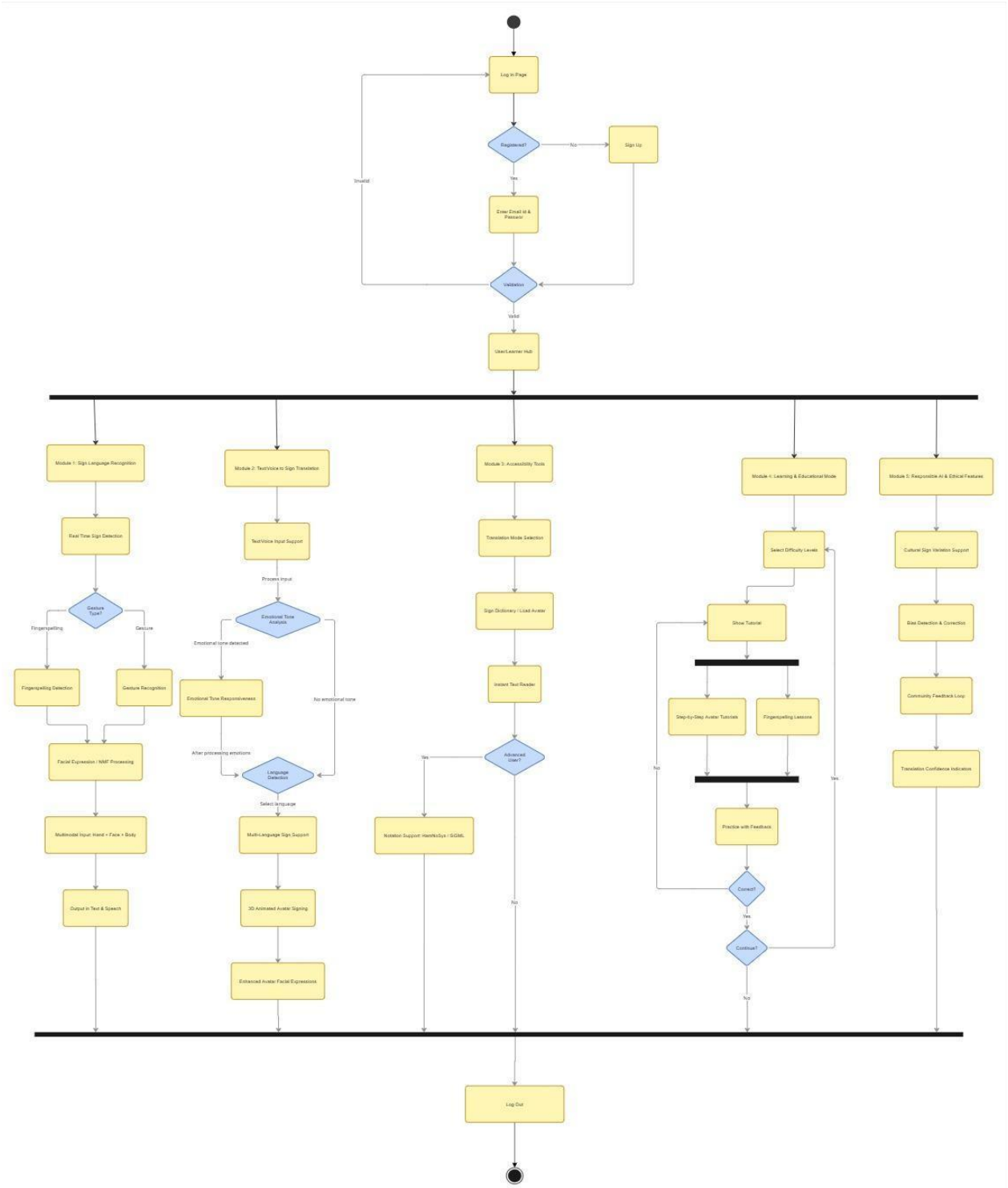
SiGML: Signing Gesture Markup Language (the code used to tell a 3D avatar how to move)

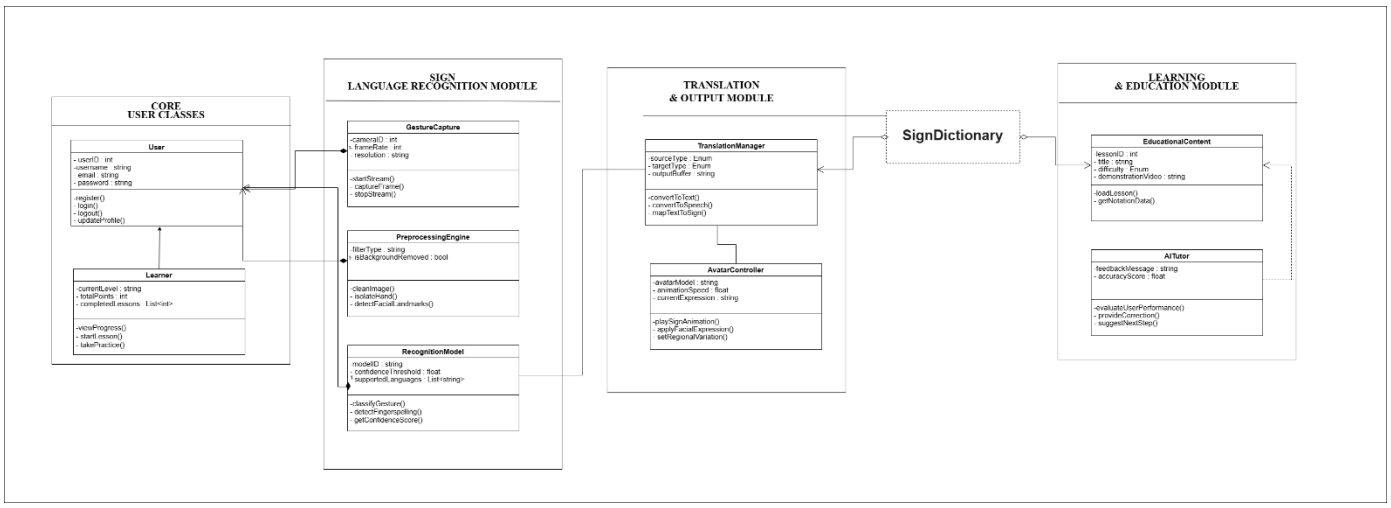
MPEG-4: Moving Picture Experts Group (a standard for high-quality video/animations)

Software Design Document

Use Case Diagram - Activity Diagram - Class Diagram


Gestura AI : Sign Language Translation & Learning System
Real Time Sign Detection
Gesture Recognition
Fingerspelling Detection
Output in Text & Speech
3D Animated Avatar Signing
Multimodal Input (Hand)
Multimodal Input (Face)
Multimodal Input (Body)
Enhanced Avatar Facial Expressions
Emotional Tone Responsiveness (Optional)
Instant Text Reader
Text Input Support
Voice Input Support
Facial Expression
NMF Processing
Offline Mode
Translation Mode Selection
Customizable Avatars
Multi-Language Sign Support (ASL/BSL/PSL etc.)
Sign Dictionary
Community Feedback Loop
Data Privacy Protections
Translation Confidence Indicators
Cultural Sign Variation Support
All Users
Step-by-Step Avatar Tutorials
Fingerspelling Lessons
Practice with Feedback
Difficulty Levels
AI Tutor Mode
Notation Support (HamNoSys / SiGML)
Learners
Advance Users
Bias Detection & Correction
System Level


Log In Page
Registered?
No
Sign Up
Yes
Enter Email id & Password
Invalid
Validation
Valid
User/Learner Hub
Module 1: Sign Language Recognition
Module 2: Text/Voice to Sign Translation
Module 3: Accessibility Tools
Module 4: Learning & Educational Mode
Module 5: Responsible AI & Ethical Features
Real Time Sign Detection
Text/Voice Input Support
Translation Mode Selection
Select Difficulty Levels
Cultural Sign Variation Support
Gesture Type?
Fingerspelling
Gesture
Process Input
Sign Dictionary / Load Avatar
Show Tutorial
Bias Detection & Correction
Emotional Tone Analysis
Emotional tone detected
No emotional tone
Fingerspelling Detection
Gesture Recognition
Emotional Tone Responsiveness
Instant Text Reader
Step-by-Step Avatar Tutorials
Fingerspelling Lessons
Community Feedback Loop
Facial Expression / NMF Processing
After processing emotions
Language Detection
Advanced User?
Yes
No
Correct?
Yes
No
Translation Confidence Indicator
Select language
Multimodal Input: Hand + Face + Body
Multi-Language Sign Support
Notation Support: HamNoSys / SiGML
Practice with Feedback
Output in Text & Speech
3D Animated Avatar Signing
Continue?
Yes
No
Enhanced Avatar Facial Expressions
Log Out


CORE
USER CLASSES
User
- userID : int
-username : string
- email : string
- password : string
-register()
- login()
- logout()
- updateProfile()
Learner
-currentLevel : string
- totalPoints : int
- completedLessons : List<int>
-viewProgress()
- startLesson()
- takePractice()
SIGN
LANGUAGE RECOGNITION MODULE
GestureCapture
-cameraID : int
- frameRate : int
- resolution : string
-startStream()
- captureFrame()
- stopStream()
PreprocessingEngine
-filterType : string
- isBackgroundRemoved : bool
-cleanImage()
- isolateHand()
- detectFacialLandmarks()
RecognitionModel
-modelID : string
- confidenceThreshold : float
- supportedLanguages : List<string>
-classifyGesture()
- detectFingerspelling()
- getConfidenceScore()
TRANSLATION
& OUTPUT MODULE
TranslationManager
-sourceType : Enum
- targetType : Enum
- outputBuffer : string
-convertToText()
- convertToSpeech()
- mapTextToSign()
AvatarController
-avatarModel : string
- animationSpeed : float
- currentExpression : string
-playSignAnimation()
- applyFacialExpression()
- setRegionalVariation()
SignDictionary
LEARNING
& EDUCATION MODULE
EducationalContent
-lessonID : int
- title : string
- difficulty : Enum
- demonstrationVideo : string
-loadLesson()
- getNotationData()
AITutor
-feedbackMessage : string
- accuracyScore : float
-evaluateUserPerformance()
- provideCorrection()
- suggestNextSteps()